

■ Implementation Plan (Version 2)

Modular Monolith Architecture · Basic Auth Edition

Recruitment Tracking System (RTS) — 22-Day Sprint Plan

This document is **Version 2** of the RTS Implementation Plan. It preserves the full modular monolith architecture, folder hierarchy, sprint structure, and feature scope of Version 1, while replacing JWT authentication with Spring Security Basic Authentication, removing Flyway in favour of a manual *tablesript.sql*, eliminating the H2 test database in favour of a dedicated *rts_test* MySQL schema, and replacing all email/SMTP notification logic with in-app dashboard alerts and pending-reminder panels.

1. Executive Summary

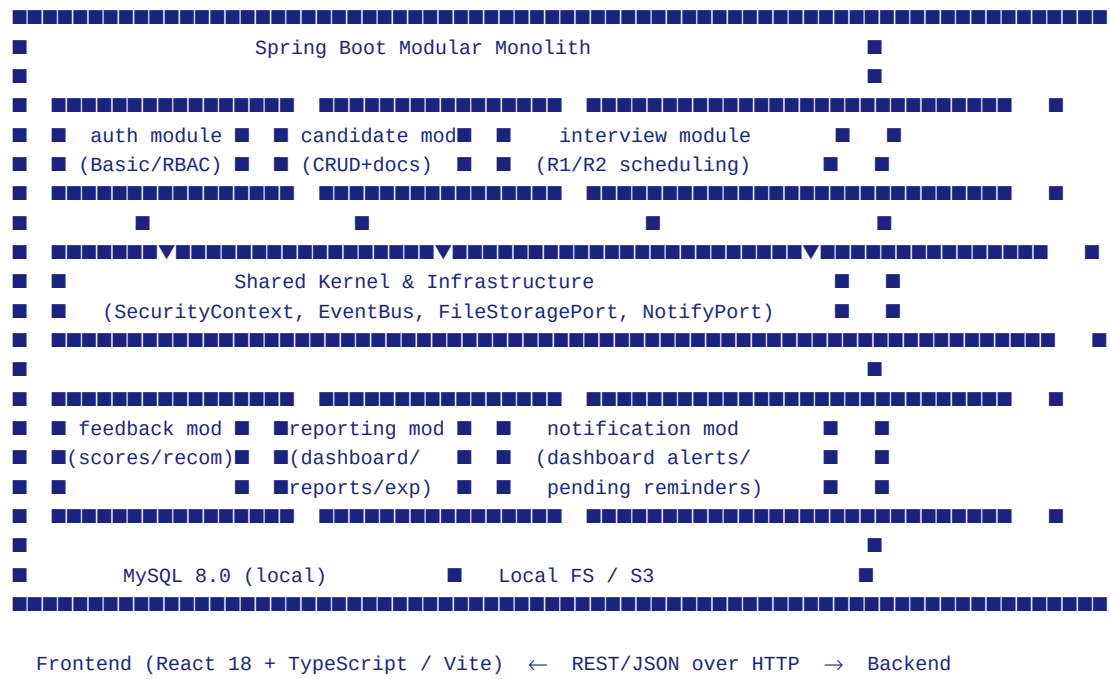
1.1 Tech Stack Overview

The Recruitment Tracking System (RTS) is built on a **Spring Boot 3.x / Java 17** backend deployed as a single Modular Monolith JAR with strict DDD-style module boundaries, and a **React 18 + TypeScript** frontend served independently via Vite. Persistence uses **MySQL 8.0** via Spring Data JPA / Hibernate with a manually maintained *tablesript.sql* for schema setup. **Spring Security HTTP Basic Authentication** handles authentication with BCrypt for password hashing (NFR-5). File storage targets the local filesystem with an optional AWS S3 adapter (SRS §6.1). Notifications are delivered exclusively through **in-app dashboard alerts, pending reminder panels, and status indicators** — no SMTP dependency. The REST API is documented with Springdoc OpenAPI / Swagger UI (NFR-19). The frontend uses **Ant Design** as the UI library, Axios for HTTP, and Redux Toolkit for global state. Vite drives the build pipeline; Jest + React Testing Library cover frontend unit tests, with JUnit 5 + Mockito on the backend.

■ **Local Environment:** All services run as direct local installations. Docker and containerisation are NOT used in this project. MySQL 8.0 must be installed and running locally on the developer's machine. Schema is initialised by running **tablesript.sql** once before first boot.

1.2 Architecture: Modular Monolith

The backend is a single deployable unit (Spring Boot JAR) structured around six self-contained feature modules that share a common kernel and infrastructure layer. Module-to-module communication occurs exclusively through well-defined application service interfaces and internal Spring application events — never through direct repository cross-calls. This enforces clear domain boundaries and enables future extraction to microservices with minimal refactoring.



1.3 Key Design Decisions

Decision	What Was Chosen	Why
Frontend Framework	React 18 + TypeScript + Ant Design	SRS §6.2; faster bootstrap than Angular for internship timeline

Decision	What Was Chosen	Why
State Management	Redux Toolkit + RTK Query	Handles async API state and caching out-of-the-box
Authentication	Spring Security HTTP Basic Auth	FR-23/NFR-5/NFR-7; simpler stateless auth for internship scope; no token management needed
ORM	Spring Data JPA + Hibernate	SRS §6.1; reduces boilerplate, supports soft-delete
DB Schema Mgmt	Manual tablescreat.sql + ddl-auto=update (dev)	Simpler than migration tooling for internship scope; schema init is a one-time manual step
File Storage	Local FS (S3 adapter ready)	SRS §6.1; local is sufficient for internship scope
API Docs	Springdoc OpenAPI / Swagger UI	NFR-19; auto-generated from annotations
Soft Delete	isDeleted flag + JPA @Where	FR-7 / US-7; audit trail retention required
Notifications	In-app dashboard alerts + pending panels + @Scheduled status checks	No SMTP dependency; all reminders surface via UI widgets on the interviewer dashboard
Database	MySQL 8.0 (local install)	Direct local installation; no Docker containers used

2. Backend Folder Structure (Modular Monolith)

2.1 Complete Tree

■ **Local Environment:** Dockerfile and docker-compose.yml are NOT included in this project. All infrastructure runs as local bare-metal installations. No Flyway, no H2, no Spring Mail dependency.

```
backend/
  pom.xml # Maven root – Spring Boot 3.x parent
  # Dependencies: mysql-connector-j (NOT h2 / NOT flyway / NOT spring-mail)
  tablescript.sql # Manual schema init – run once before first boot
  .env.example # Environment variable template
  .gitignore
  src/
    main/
      java/com/rtts/
        RtsApplication.java # @SpringBootApplication entry point
        shared/ # SHARED KERNEL
          kernel/
            BaseEntity.java # id, createdAt, updatedAt, isDeleted
            AuditableEntity.java # createdBy, updatedBy
            RecruitmentStage.java # Enum: APPLICATION_RECEIVED ... REJECTED
            events/
              InterviewScheduledEvent.java
              StageChangedEvent.java
              FeedbackSubmittedEvent.java
            exception/
              ResourceNotFoundException.java
              ValidationException.java
              ConflictException.java
              GlobalExceptionHandler.java # @RestControllerAdvice
            response/
              ApiResponse.java # Unified response wrapper
              PagedResponse.java
            util/
              FileValidator.java
              DateUtil.java
          infrastructure/ # INFRASTRUCTURE LAYER
            config/
              SecurityConfig.java # Basic Auth filter chain, CORS, RBAC
              FileStorageConfig.java # Upload dir, S3 profile
              SwaggerConfig.java # Springdoc OpenAPI setup
              SchedulerConfig.java # @EnableScheduling
            security/
              CustomUserDetailsService.java # Loads user from DB for Basic Auth
              BasicAuthEntryPoint.java # 401 response on missing/bad creds
            storage/
              FileStoragePort.java # Interface (hexagonal)
              LocalFileStorageAdapter.java
              S3FileStorageAdapter.java # Optional, activate via profile
            notification/
              NotificationPort.java # Interface
              InAppNotificationAdapter.java # Persists alerts to DB table
          modules/ # FEATURE MODULES
            auth/ # Module: Authentication & User Mgmt
            domain/
```

```

# User.java # @Entity extends AuditableEntity
# Role.java # Enum: ADMIN, HR_MANAGER, RECRUITER, INTERVIEWER
# application/
# AuthService.java # login(), getAuthenticatedUser()
# UserService.java # getProfile(), updateProfile()
# persistence/
# UserRepository.java # JpaRepository<User, UUID>
# api/
# AuthController.java # POST /api/auth/login (verify + return user info)
# UserController.java # GET/PUT /api/users/profile
# dto/
# LoginRequest.java
# LoginResponse.java # username + role (no token field)
# UserProfileDto.java

# candidate/ # Module: Candidate Management
# domain/
# Candidate.java # @Entity: name, email, phone, position, stage
# StageHistory.java # @Entity: stage, changedAt, changedBy
# CandidateDocument.java # @Entity: type, filePath, linkedAt
# application/
# CandidateService.java # create, read, update, softDelete, paginate
# StageService.java # updateStage(), bulkUpdate(), getHistory()
# DocumentService.java # uploadResume(), uploadPhoto()
# persistence/
# CandidateRepository.java
# StageHistoryRepository.java
# CandidateDocumentRepository.java
# api/
# CandidateController.java # CRUD endpoints
# StageController.java # PUT /stage, POST /bulk-stage, GET /stage-history
# DocumentController.java # POST /resume, POST /photo
# dto/ (CreateCandidateDto, CandidateResponseDto, StageUpdateDto ...)

# interview/ # Module: Interview Scheduling
# domain/
# Interview.java # round, dateTime, interviewers, status
# InterviewPhoto.java # filePath, caption
# application/
# InterviewService.java # scheduleR1, scheduleR2, reschedule, cancel
# persistence/
# InterviewRepository.java
# InterviewPhotoRepository.java
# api/
# InterviewController.java
# dto/ (ScheduleInterviewDto, InterviewResponseDto ...)

# feedback/ # Module: Feedback Collection
# domain/
# Feedback.java # ratings, comments, recommendation, submittedAt
# application/
# FeedbackService.java # submit(), getByCandidate(), getPending()
# persistence/
# FeedbackRepository.java
# api/
# FeedbackController.java # POST /api/feedback, GET /api/candidates/{id}/feedback
# dto/ (FeedbackSubmitDto, FeedbackResponseDto ...)

# reporting/ # Module: Reporting & Analytics
# application/
# ReportingService.java # dashboard(), stageReport(), timeToHire(), interviewer()

```

```

■ ■ ■ ■■■ api/
■ ■ ■ ■■■ DashboardController.java # GET /api/dashboard/metrics
■ ■ ■ ■■■ ReportController.java # GET /api/reports/*
■ ■ ■ ■■■ dto/ (DashboardMetricsDto, StageReportDto ...)
■ ■ ■
■ ■ ■■■ notification/ # Module: In-App Alerts & Reminders
■ ■ ■■■ domain/
■ ■ ■ ■■■ Notification.java # @Entity: userId, message, type, isRead, createdAt
■ ■ ■■■ application/
■ ■ ■■■ NotificationService.java # Listens to domain events, @Scheduled pending check
■ ■
■ ■■■ resources/
■ ■■■ application.yml # Base config (datasource → MySQL, storage)
■ ■■■ application-dev.yml # Dev overrides (local MySQL rts_dev, ddl-auto=update)
■ ■■■ application-test.yml # Test profile: rts_test MySQL schema
■ ■■■ application-prod.yml # Prod overrides (env-var driven)

```

2.1.1 pom.xml — Dependency Notes

Add the MySQL connector. Do NOT include Flyway, H2, or Spring Mail starters:

```

<!-- MySQL Connector -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Spring Security (Basic Auth – no JWT library needed) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- X DO NOT ADD: spring-boot-starter-mail -->
<!-- X DO NOT ADD: flyway-core -->
<!-- X DO NOT ADD: h2 (even test scope) -->
<!-- X DO NOT ADD: io.jsonwebtoken / jjwt -->

```

2.1.2 tablesript.sql — Schema Setup

■ **Note:** Run tablesript.sql manually against your local MySQL instance before first boot. Use `spring.jpa.hibernate.ddl-auto=update` in `application-dev.yml` for development convenience. Never use create-drop in production.

```

-- tablesript.sql (run once: mysql -u root -p rts_dev < tablesript.sql)
CREATE DATABASE IF NOT EXISTS rts_dev CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE rts_dev;

CREATE TABLE users (
  id          VARCHAR(36) NOT NULL PRIMARY KEY,
  username    VARCHAR(100) NOT NULL UNIQUE,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  role        VARCHAR(50) NOT NULL,
  is_deleted  TINYINT(1) NOT NULL DEFAULT 0,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Additional tables: candidates, stage_history, candidate_documents,

```

```
-- interviews, interview_photos, feedback, notifications
-- (full DDL provided in tablesript.sql in the project repository)
```

2.2 Module Communication Rules

- **auth** → **candidate/interview/feedback**: AuthService exposes only *getAuthenticatedUser()*. No repository cross-calls.
- **candidate** → **interview**: Interview module listens to *StageChangedEvent* published via Spring ApplicationEventPublisher.
- **interview** → **notification**: *InterviewScheduledEvent* triggers NotificationService via @EventListener — creates an in-app alert record.
- **feedback** → **notification**: @Scheduled job in NotificationService queries FeedbackRepository for interviews with no feedback after 24h and creates pending-reminder notification records.
- **feedback** → **reporting**: ReportingService queries FeedbackRepository directly (same bounded context, reporting is read-side only).
- No module imports another module's repository. Domain objects shared only via DTOs through service interfaces.

3. Frontend Folder Structure

- **Local Environment:** Dockerfile is NOT included in the frontend directory. The dev server runs directly via `npm run dev` (Vite). No containerised build step.

[illegible]

[illegible]

■ **Note:** The `authSlice.ts` stores username + role only. Axios interceptor attaches the `Authorization: Basic` header on every request using credentials held in Redux state (or browser `sessionStorage`). No JWT token field exists anywhere in the frontend.

4. Sprint 1 Implementation Plan (Days 1–11)

Sprint 1 Goal: Deliver a fully authenticated, role-secured system with complete Candidate CRUD, file uploads (resume + photo), and Stage Tracking with history. All APIs documented and unit-tested. Authentication is Spring Security Basic Auth — no JWT, no Flyway, no H2, no email.

Day	Story IDs	Tasks & Activities	Modules Touched	Deliverables	Hrs	Risk
Day 1	T0, T1	Initialize Spring Boot 3.x project (pom.xml with mysql-connector-j, module skeleton), React 18 + Vite scaffold. Run tablesript.sql to create rts_dev schema (users, candidates, stage_history tables). Create Git repo with main/develop branches. Configure Spring Security Basic Auth: SecurityConfig, CustomUserDetailsService, BasicAuthEntryPoint (T1). Seed initial admin user directly into DB.	Infrastructure, Auth module	Runnable Spring Boot app, MySQL connected locally via tablesript.sql, React dev server running, Basic Auth returns 401 on unauthenticated request	11	Med
Day 2	US-24: T2, T3	Implement POST /api/auth/login (T2): validate credentials via Basic Auth, return user info + role, structured 401 on failure. Build React login page (T3): username/pwd form, remember-me toggle, credentials sent as Basic header, dashboard redirect on success.	Auth/api, Auth/domain, Auth frontend	Working login endpoint (Postman verified), Login UI renders, Role returned on success	7	Low
Day 3	US-24: T4 US-25: T5	Unit test login flow (T4): valid/invalid credentials, 401 response, role in response body. Define RBAC rules (T5): ADMIN, HR_MANAGER, RECRUITER, INTERVIEWER roles; configure Spring Security method-level @PreAuthorize at service layer.	Auth/application, SecurityConfig	Login unit tests passing (≥4 cases), RBAC annotations on all secured methods	5	Low
Day 4	US-25: T6 US-26: T6a US-1: T7	Implement React route guards + API 403 interceptor (T6). Build GET/PUT /api/users/profile with password-change validation (T6a). Implement POST /api/candidates with UUID generation and validation (T7).	Auth frontend, Candidate/api, Candidate/application	Protected routes enforce roles, Profile API tested, Candidate creation endpoint live	10	Med
Day 5	US-26: T6b US-1: T8 US-2: T10	Build profile management page (T6b): editable fields, photo upload, save/cancel. Build candidate registration form (T8): mandatory fields, email/phone validation, submission. Implement POST /api/candidates/{id}/resume: PDF/DOC/DOCX + 5MB validation (T10).	Auth frontend, Candidates/pages, Documents/api	Profile page functional, Candidate form submits successfully, Resume upload stores file	10	Med

Day	Story IDs	Tasks & Activities	Modules Touched	Deliverables	Hrs	Risk
Day 6	US-1: T9 US-2: T11 US-3: T12	Unit test candidate creation (T9): field validation, ID generation, confirmation. Build resume upload UI with replace-file option and error messages (T11). Implement photo upload endpoint (T12): JPG/PNG, 2MB, compression.	Candidate tests, Documents frontend, Documents/api	Candidate unit tests green, Resume upload UI complete, Photo endpoint validated	9	Low
Day 7	US-3: T13, T14 US-4: T15	Build photo upload UI (T13): file picker, thumbnail preview, full-size view. Unit tests for resume+photo (T14): format/size validation, replace behaviour. Implement paginated GET /api/candidates with sort + filter query params (T15).	Documents frontend, Candidate/persistence	Photo upload UI complete, Upload unit tests passing, Candidate list API returns paginated data	9	Low
Day 8	US-4: T16a, T16b US-5: T18	Build candidate list table UI (T16a): Photo/Name/Position/Stage/Date columns, pagination, sortable headers. Add search bar + filter panel (T16b): name/email search, stage/position/date-range filters. Implement GET /api/candidates/{id} full profile (T18).	Candidates/pages, Candidate/api	Candidate list renders with pagination, Search + filters working, Detail endpoint returns full data	9	Med
Day 9	US-4: T17 US-5: T19, T20 US-7: T24	Unit test candidate list (T17): pagination/filter/sort/search. Build candidate detail page (T19): profile section, resume link, photo, stage badge, interview summary. Unit test detail page (T20). Implement soft-delete (T24): isDeleted flag, exclude from queries.	Candidates tests, Candidates/pages, Candidate/application	List unit tests green, Detail page renders fully, Soft-delete endpoint working	15	Med
Day 10	US-6: T21, T22, T23 US-7: T25 US-8: T27	Implement PUT /api/candidates/{id} (T21): validate, persist, record updatedAt. Build edit form (T22): prefilled fields, save/cancel. Unit test edit (T23). Add delete action with confirmation dialog (T25). Implement PUT /api/candidates/{id}/stage with history auto-log (T27).	Candidate/api, Candidates/pages, Stage/application	Edit endpoints and form functional, Delete confirmation dialog works, Stage update persists with history	16	High
Day 11	US-7: T26 US-8: T28, T29 US-9: T30, T31 T32, T32b	Unit test soft-delete (T26). Build stage dropdown + confirm dialog UI (T28). Unit test stage update (T29). Implement GET /api/candidates/{id}/stage-history (T30). Build stage timeline view (T31). Sprint 1 integration test (T32): auth→CRUD→uploads→stage E2E. Bug fixes + code review (T32b).	All S1 modules, Integration tests	All Sprint 1 features passing integration tests, Stage history timeline renders, Code reviewed and merged	24	High

5. Sprint 2 Implementation Plan (Days 12–22)

Sprint 2 Goal: Deliver Interview Scheduling (Round 1 + Round 2), Feedback Collection with in-app reminder dashboard, Reporting & Analytics, and full data export. All Sprint 1 features must be stable before Day 12. **No email/SMTP tasks exist in Sprint 2** — feedback reminders are handled exclusively by the @Scheduled NotificationService writing in-app alert records, surfaced via the PendingFeedbackPanel and NotificationBell components.

■ **Note:** Sprint 1 Prerequisite: Auth (Basic Auth), Candidate CRUD, File Uploads, and Stage Tracking must be merged, tested, and stable.

Day	Story IDs	Tasks & Activities	Modules Touched	Deliverables	Hrs	Risk
Day 12	US-10: T33 US-11: T36	Implement POST /api/candidates/bulk-stage (T33): accept list of IDs + target stage, persist, log individual history per candidate. Implement POST /api/interviews/round1 (T36): schedule conflict validation, create interview record, trigger InterviewScheduledEvent, auto-update candidate stage to 'Round 1 Scheduled', create in-app notification via NotificationService.	Candidate/stage, Interview/api, Notification	Bulk stage endpoint working, Round 1 scheduling endpoint live, In-app notification created on schedule	8	Med
Day 13	US-10: T34, T35 US-11: T37	Build bulk stage UI (T34): multi-select checkboxes, target dropdown, count confirmation. Unit test bulk stage (T35). Build Round 1 scheduling form (T37): candidate picker, date/time, multi-interviewer, meeting link, duration (30/45/60 min), optional notes.	Candidates frontend, Interview frontend	Bulk stage UI functional with count confirmation, R1 scheduling form renders and submits	10	Med
Day 14	US-11: T38 US-12: T39	Unit test R1 scheduling (T38): conflict detection, stage auto-update, notification creation. Implement POST /api/interviews/round2 (T39): enforce Round 1 cleared prerequisite check, location field, conflict validation, fire notification.	Interview/api, Interview tests	R1 unit tests passing, Round 2 endpoint live with prerequisite guard	8	Med
Day 15	US-12: T40, T41 US-13: T42	Build Round 2 face-to-face form (T40): candidate filter (R1 cleared only), location/room field, duration, notes. Unit test R2 scheduling (T41): prerequisite enforcement, conflict check. Implement GET /api/interviews/schedule with date-range + interviewer filters (T42).	Interview frontend, Interview/api	R2 form works with prerequisite filter, R2 unit tests green, Schedule list API verified	9	Low

Day	Story IDs	Tasks & Activities	Modules Touched	Deliverables	Hrs	Risk
Day 16	US-13: T43a, T43b, T44 US-14: T45	Build interview calendar UI (T43a): day/week/month view, colour-coded R1 vs R2, click-to-detail. Add iCal export (T43b): generate .ics file download. Unit test calendar (T44). Implement PUT /api/interviews/{id}/reschedule (T45): conflict check, update record, update in-app notification, log history.	Interview frontend, Interview/api	Calendar renders with colour coding, iCal export working, Reschedule endpoint live	12	Med
Day 17	US-14: T46, T47 FR-15: T48, T49 US-15: T57	Build reschedule form (T46): pre-filled time, reason field, conflict indicator. Unit test reschedule (T47). Implement cancel endpoint PATCH (T48): canceled status, stage rollback. Add cancel button to calendar+detail (T49). Implement multi-photo upload endpoint (T57): JPG/PNG 5MB, max 10/interview.	Interview frontend, Interview/api, Documents	Reschedule form works, Cancel flow complete, Photo upload endpoint validates files	14	High
Day 18	FR-15: T50 US-15: T58 US-16: T51	Unit test cancel (T50): status, stage rollback, reason saved. Build photo gallery UI (T58): multi-file upload, gallery view, captions, delete option. Implement POST /api/feedback (T51): validate all 5 ratings, store, update eval score, enforce 24h edit lock.	Interview tests, Interview frontend, Feedback/api	Cancel unit tests green, Photo gallery UI complete, Feedback submission endpoint live	10	Med
Day 19	US-16: T52, T53 US-17: T54 US-18: T59	Build structured feedback form (T52): 5 rating sliders (1-5), 1000-char textarea, Select/Reject/Hold/Proceed dropdown. Unit test feedback submission (T53). Implement GET /api/candidates/{id}/feedback with averages (T54). Implement @Scheduled pending-feedback checker (T59): runs every hour, writes Notification records for interviewers with no feedback 24h after interview (no email — in-app only).	Feedback frontend, Feedback/api, Notification	Feedback form submits correctly, Feedback unit tests green, Reminder scheduler writes in-app notifications	13	Med
Day 20	US-17: T55, T56 US-18: T59b US-19: T60	Build feedback summary view (T55): round-wise cards, average scores, colour-coded recommendations (green/red/yellow), printable. Unit test feedback display (T56). Add PendingFeedbackPanel to interviewer dashboard (T59b): shows list of overdue feedback items with click-to-form links. Implement GET /api/dashboard/metrics (T60): total candidates, stage distribution, 7-day recents, this-week interviews, selection rate.	Feedback frontend, Reports/api	Feedback summary renders with correct colour coding, Pending panel shows overdue items, Dashboard API returns all metrics	13	Med

Day	Story IDs	Tasks & Activities	Modules Touched	Deliverables	Hrs	Risk
Day 21	US-19: T61, T62 US-20: T63, T64, T65 US-21: T66 US-22: T68 US-23: T72	Build dashboard UI (T61): KPI cards, bar/pie chart (Recharts), auto-refresh every 5 min, NotificationBell in header. Unit test dashboard (T62). Implement stage distribution report endpoint (T63) + UI with drill-down + export (T64). Unit tests (T65). Implement time-to-hire endpoint (T66). Implement interviewer performance endpoint (T68). Implement candidate export endpoint (T72): full Excel file.	Reports/api, Reports frontend	Dashboard live with charts and notification bell, Stage report with export, Time-to-hire and interviewer endpoints ready, Export endpoint generates Excel	20	High
Day 22	US-21: T67 US-22: T69 US-23: T73, T74 T75, T76, T77, T78	Build time-to-hire report UI (T67): avg days KPI, position table, longest/shortest highlights, export. Build interviewer performance UI (T69): stats table, recommendation bar, pending indicator, Excel export. Add export button to candidate list (T73). Unit test export (T74). Full integration test (T75): all Sprint 2 flows. Bug fixes (T76). Sprint 1 regression test (T77). Full system E2E test (T78).	Reports frontend, All modules (regression)	All reports functional with export, System E2E test passing, Application deployment-ready	25	High

6. Cross-Cutting Concerns & Best Practices

6.1 Authentication & Authorization

- Spring Security stateless filter chain with HTTP Basic Authentication (RFC 7617). Credentials validated on every request against the users table via CustomUserDetailsService.
- BCrypt password encoding with strength 12 (NFR-5). Passwords never stored in plaintext.
- Four roles enforced via @PreAuthorize at the service layer: **ADMIN, HR_MANAGER, RECRUITER, INTERVIEWER** (FR-24).
- React ProtectedRoute reads role from Redux store (set on login). Unauthorised access redirects to /403 with a clear message (US-25).
- Axios interceptor attaches Authorization: Basic <base64(user:pass)> header on every outbound API call.
- Session timeout after 2 hours of inactivity enforced client-side by clearing credentials from Redux state (US-24 AC). Server-side: Spring Security re-validates on each request, so stale or cleared credentials result in a 401.

6.2 Notification Strategy (In-App Only)

- No SMTP, no Spring Mail, no Mailtrap. Notifications are persisted as Notification entity records in the notifications MySQL table.
- The NotificationService listens to domain events (InterviewScheduledEvent, StageChangedEvent, FeedbackSubmittedEvent) via @EventListener and creates Notification records.
- A @Scheduled job runs hourly: it queries FeedbackRepository for interviews with no submitted feedback 24+ hours post-interview and writes pending-reminder Notification records for the assigned interviewers.
- The frontend polls GET /api/notifications every 60 seconds. NotificationBell in AppLayout shows the unread count badge. NotificationDrawer lists all alerts with read/dismiss actions.
- PendingFeedbackPanel on the interviewer's dashboard provides a direct action list with click-to-form links for overdue feedback items.

6.3 Error Handling & Logging

- GlobalExceptionHandler (@RestControllerAdvice) maps all exceptions to structured ApiResponse: { success, message, data, timestamp }.
- Standard HTTP codes: 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict (scheduling).
- SLF4J + Logback with JSON output in production (for log aggregation); console in dev.
- Frontend: Axios response interceptor; toast notifications via Ant Design message API; ErrorBoundary for runtime crashes.

6.4 Testing Strategy

Layer	Tool	What Is Tested	Target Coverage	When Run
Backend Unit	JUnit 5 + Mockito	Service logic, validators, Basic Auth, soft-delete	≥70% line coverage	Every day (CI)
Backend Integ.	@SpringBootTest + dedicated rts_test MySQL schema	Controller→Service→DB round-trips	All critical paths	Day 11, Day 22
Frontend Unit	Jest + React Testing Library	Components, hooks, form validation	Key components	Every day (CI)
API Testing	Postman collection	All REST endpoints, edge cases (Basic Auth header required)	100% endpoints	End of each day

Layer	Tool	What Is Tested	Target Coverage	When Run
E2E (Manual)	Browser walk-through	Full recruitment workflow (US-24 → US-23)	Success criteria §7	Day 22 (T78)

■ **Note:** Integration Test Configuration: Create a dedicated `rts_test` MySQL schema (`CREATE DATABASE rts_test;`) and configure `application-test.yml` to point to it with `spring.jpa.hibernate.ddl-auto=create-drop`. Run `tablesript.sql` before the test suite starts. This guarantees full MySQL dialect compatibility.

6.5 Database Schema Management

- Schema is initialised by running **tablesript.sql** once on a fresh MySQL instance: `mysql -u root -p rts_dev < tablesript.sql`
- For development: `spring.jpa.hibernate.ddl-auto=update` in `application-dev.yml` lets Hibernate add columns automatically during feature development.
- For production: `ddl-auto=validate`. Schema changes require manually updating `tablesript.sql` and re-running changed statements.
- Never use `ddl-auto=create` or `create-drop` outside of the test profile.
- Daily `mysqldump` backups in production (NFR-15): `mysqldump -u root -p rts_prod > rts_backup_$(date +%F).sql`

6.6 Environment Configuration

Setting	Dev	Staging	Prod
Database	Local MySQL 8.0 (rts_dev)	Local MySQL 8.0 (rts_staging)	MySQL 8.0 (RDS or VPS)
Notifications	In-app only (DB records)	In-app only (DB records)	In-app only (DB records)
File Storage	Local FS (/uploads)	Local FS	Local FS or AWS S3
CORS	<code>http://localhost:5173</code>	<code>https://staging.rts.com</code>	<code>https://rts.com</code>
Auth	Basic Auth (dev-user seeded in <code>tablesript.sql</code>)	Basic Auth	Basic Auth (env-var credentials)

6.7 Code Quality Tools

- Backend: Checkstyle (Google style), SpotBugs, JaCoCo coverage gate ($\geq 70\%$).
- Frontend: ESLint (react + typescript-eslint), Prettier, Husky pre-commit hooks.
- Pre-commit hook runs: `eslint --fix + prettier --write + mvn checkstyle:check`.

6.8 Version Control Branching Strategy

- `main` ← production-ready code only (protected)
- `develop` ← integration branch
- `feature/US-xx` ← one branch per user story, merged to `develop` via PR
- `hotfix/xxx` ← direct to `main` + cherry-pick to `develop`
- PRs require at least 1 review. Squash merge to `develop`. Merge commit to `main`.
- Commit message format: `feat(US-1): add candidate registration endpoint`

7. Risks, Assumptions & Mitigation Plan

#	Risk	Level	Impact	Sprint	Mitigation
R1	Day 10–11 overload (Edit + Delete + Stage all due together)	High	Sprint 1 may not complete on Day 11	S1	Start T27 (stage) on Day 10 in parallel. Defer T31 (timeline UI) to Day 12 if needed.
R2	tablescript.sql inconsistency across team members (MySQL version differences)	Med	Schema divergence between machines	S1	Document exact MySQL 8.0 install steps. Version-control tablescript.sql. Run it from Day 1.
R3	Day 21 has 5 concurrent tasks (reports + export)	High	Day 22 becomes pure bug-fix debt	S2	Implement report endpoints (T63, T66, T68, T72) on Day 20 evening. Day 21 focuses on UI only.
R4	File upload validation bypassed — malicious file uploaded	High	Security vulnerability (NFR-8/9)	S1	FileValidator.java enforces MIME type + extension + size on Day 5. Unit tests (T14) verify all edge cases.
R5	Round 2 prerequisite (R1 cleared) not enforced	Med	Business rule violated (US-12 AC)	S2	Dedicated guard in InterviewService.scheduleRound2() + T41 unit test asserts rejection when R1 not cleared.
R6	Feedback 24h lock not enforced at scale	Low	Interviewers edit feedback (US-16 AC violation)	S2	DB field submittedAt set on first POST. FeedbackService checks age before allowing update. T53 validates.
R7	Schedule conflict detection race condition	Med	Two overlapping interviews booked for same interviewer	S2	Use DB-level @Version optimistic lock + service-layer conflict window check (interviewer × time window).
R8	In-app notifications not read by interviewers (no email push)	Med	Feedback reminders missed; evaluations not completed on time	S2	NotificationBell badge is always visible in header. PendingFeedbackPanel on dashboard is prominent. T59b verifies panel renders overdue items.

7.1 Assumptions

- Team size: 4 members (Member 1–4) working full-time as assigned in the Sprint Backlog.
- Database choice: MySQL 8.0, installed locally on each developer's machine and on the staging server. Docker is not used.
- Notifications: All reminders and alerts are delivered exclusively in-app. No SMTP credentials are required at any stage of development.
- Frontend framework choice: React 18 (chosen over Angular for faster setup within the 22-day window).
- No external calendar integration beyond iCal export (FR-13 / US-13 AC: Google/Outlook via .ics file).
- File storage: Local filesystem for the internship scope; S3 adapter is coded but not activated.

- `tablesript.sql` is run manually once per environment (dev, staging, prod). No automated migration tool is used.

8. Final Deliverables Checklist

8.1 End of Day 11 (Sprint 1) — Must Be Complete

✓	Deliverable	Acceptance / Ref	Status
■	Spring Security Basic Auth login with 2-hour client-side session timeout and BCrypt passwords	US-24, NFR-5/6/7	To Do
■	RBAC enforced for Admin, HR Manager, Recruiter, Interviewer roles	US-25, FR-24	To Do
■	User profile page with editable fields and photo upload	US-26	To Do
■	Candidate registration form with all mandatory fields and unique ID generation	US-1, FR-1	To Do
■	Resume upload (PDF/DOC/DOCX, max 5MB) linked to candidate profile	US-2, FR-2	To Do
■	Photo upload (JPG/PNG, max 2MB) with thumbnail in list view	US-3, FR-3	To Do
■	Paginated candidate list (20/page) with search, sort, and filters	US-4, FR-4	To Do
■	Candidate detail page showing all info, resume link, photo, stage	US-5, FR-5	To Do
■	Edit candidate form (all fields except ID) with save confirmation	US-6, FR-6	To Do
■	Soft-delete with confirmation dialog; deleted records excluded from lists	US-7, FR-7	To Do
■	Stage update with dropdown, confirmation, and auto-logged history	US-8, FR-8	To Do
■	Stage history timeline view with timestamps and user attribution	US-9, FR-9	To Do
■	All Sprint 1 unit tests passing ($\geq 70\%$ coverage)	T4/9/14/17/20/23/26/29	To Do
■	Integration test T32 passing (end-to-end auth → CRUD → stage) using rts_test MySQL schema	T32	To Do
■	Swagger UI accessible at /swagger-ui.html with all Sprint 1 endpoints	NFR-19	To Do

8.2 End of Day 22 (Sprint 2) — Must Be Complete (+ All Sprint 1 Items Regression-Clear)

✓	Deliverable	Acceptance / Ref	Status
■	Bulk stage update for multiple candidates with count confirmation	US-10	To Do
■	Round 1 online interview scheduling with meeting link and in-app notification	US-11, FR-10	To Do
■	Round 2 face-to-face scheduling restricted to Round 1 cleared candidates only	US-12, FR-11	To Do

✓	Deliverable	Acceptance / Ref	Status
■	Interview calendar with day/week/month view, colour coding, and iCal export	US-13, FR-13	To Do
■	Reschedule and cancel interview with in-app notification update and history	US-14, FR-14/15	To Do
■	Interview photo gallery (max 10 photos with captions)	US-15, FR-12	To Do
■	Structured feedback form (5 ratings + recommendation) with 24h lock	US-16, FR-16/17	To Do
■	Candidate feedback summary with averages and colour-coded recommendations	US-17, FR-18	To Do
■	@Scheduled pending-feedback checker writes in-app Notification records 24h post-interview; PendingFeedbackPanel surfaces them on interviewer dashboard	US-18	To Do
■	Recruitment dashboard with KPI cards, stage chart, notification bell, 5-min auto-refresh	US-19, FR-19	To Do
■	Stage distribution report with drill-down and PDF/Excel export	US-20, FR-20	To Do
■	Time-to-hire report with position breakdown and export	US-21, FR-21	To Do
■	Interviewer performance report with recommendation distribution	US-22	To Do
■	Candidate data export to Excel (all fields, file paths for docs)	US-23, FR-22	To Do
■	Full system E2E test T78 passing: login → candidate → schedule → feedback → report	T78, SRS §7	To Do
■	Sprint 1 regression test T77 passing (no regressions introduced in Sprint 2)	T77	To Do
■	Application deployed and accessible (SRS §7.10) with Swagger UI updated	SRS §7	To Do